

nanogt Dissertation Research Brief

MotifPeeker Analysis | Literature Review | Scope Guidance

Lili Yassin | MRes Bioinformatics | Imperial College London | June 2026

1. MotifPeeker — What Is It and How Does It Compare?

1.1 What MotifPeeker Does

MotifPeeker is an R/Bioconductor package built by Hiranyamaya (Hiru) Dash in the Neurogenomics Lab at Imperial College London. Its sole purpose is benchmarking epigenomic profiling methods (ChIP-seq, TIP-seq, CUT&Tag, CUT&Run) by using motif enrichment as the main quality metric. Given a set of peak files from different sequencing methods, it produces a self-contained HTML report covering three analyses: general quality metrics (peak widths, FRiP scores, motif-to-summit distances), known motif enrichment (how often user-supplied motifs appear in peaks), and de novo motif discovery (finding new motifs and matching them to JASPAR).

It wraps the MEME suite (a major external dependency), integrates ENCODE and JASPAR databases directly, and uses Bioconductor's BiocParallel for parallelism. The entire codebase is in R (~99%), packaged as a formal Bioconductor package with automated CI/CD, unit tests, vignettes, Docker containers, and a pkgdown documentation website. It was published in Bioinformatics Advances (Oxford) in 2026 — doi:10.1093/bioadv/vbag023.

1.2 Complexity and Technical Demands

MotifPeeker is technically demanding. It requires: deep knowledge of R package development conventions (DESCRIPTION, NAMESPACE, roxygen2 docs); familiarity with Bioconductor data structures (GRanges, BamFile); integration of a non-trivial external tool (MEME suite); parallelism via BiocParallel; HTML report generation from R Markdown; and a full CI/CD pipeline with Codecov and GitHub Actions. The repo has 183 commits, multiple contributors, and was clearly iterated over months.

Honest complexity rating

MotifPeeker is a polished, publication-ready R package. It is more engineering-heavy than biology-heavy. Building it required strong R package development skills, not just scripting. The biology (motif enrichment) is well-established — the novelty is in wrapping it cleanly into a benchmarking framework.

1.3 How Similar Is It to nanogt?

Structurally, MotifPeeker and nanogt are more similar than they first appear. Both are computational tools that take biological data as input, run a multi-step analysis pipeline, and

produce a structured report. Both integrate external databases. Both are designed to be reusable tools rather than one-off analyses. However, the biology is completely different: MotifPeeker is epigenomics/chromatin; nanogt is gene therapy/regulatory science. The comparison is most useful at the engineering level — how to structure a bioinformatics tool, what a published version looks like, what level of testing and documentation is expected.

Key lessons to steal from MotifPeeker: automated HTML/report output; a proper CLI; integration of real external databases (you use Orphanet and UniProt, which is the equivalent); a clear separation between data fetching, scoring, and reporting; unit tests; and a dedicated documentation site or at least a clean README.

2. Core Literature — Papers You Must Read

These papers are directly relevant to nanogt. For each, I have noted which section(s) of their paper are worth stealing, and what their limitations are that you could address.

2.1 PaVe-GT: NIH Platform Vector Gene Therapy Program

The single most relevant real-world project to your dissertation. PaVe-GT (Platform Vector Gene Therapy) is an NIH/NCATS initiative doing exactly what nanogt is trying to formalise computationally: using a platform AAV vector approach across multiple rare diseases to share regulatory knowledge, manufacturing processes, and clinical precedents. More than 80% of rare diseases are monogenic and amenable to gene therapy; PaVe-GT explicitly tests whether one disease's approved GT program can inform development for another.

Why this matters to you

PaVe-GT is the institutional precedent for your entire project premise. Cite it early. Their 2025 paper (Lomash et al., Human Gene Therapy, doi:10.1089/hum.2024.230) covers AAV development pathway, early regulatory planning, and how they chose which diseases to include. Their Target Product Profile (TPP) documents are publicly available and structurally map almost directly onto your 6-dimension scoring algorithm.

- What to steal: Their disease selection criteria, how they define a 'platform', how they justify choosing related diseases, their regulatory strategy section
- Limitation you address: PaVe-GT is manual and expert-driven — there is no computational tool to extend this to the 7,000+ other rare diseases. nanogt fills that gap.
- Key URL: <https://pave-gt.ncats.nih.gov/> — read all Resources and Publications
- [Lomash et al. 2025 — AAV Gene Therapy Development: Early Planning and Regulatory Considerations](#) — The PaVe-GT methodology paper. Read the full methods and regulatory strategy.
- [Brooks et al. 2020 — The Platform Vector Gene Therapies Project](#) — The original PaVe-GT concept paper. Cites the efficiency rationale.

2.2 AAV Serotype Selection and Tropism

Your scoring algorithm presumably includes an AAV serotype/tissue tropism dimension. This literature gives you the data and justification for how those scores are defined.

- [Comprehensive Atlas of AAV Tropism \(2025, Molecular Therapy\)](#) — Systematic study of 10 AAV serotypes across tissues in mice. Use this to justify tissue tropism scores in your algorithm.
- [AAV as a delivery vector — Signal Transduction & Targeted Therapy \(Nature, 2024\)](#) — Comprehensive review of AAV biology, serotypes, tropism, and clinical use. Essential background.
- [AAV Serotypes and Tissue Tropism — Addgene Blog](#) — Clear explainer on serotype-tropism relationships. Good for your Introduction.
- What to steal: Tables mapping AAV serotypes to target tissues. Convert these into your scoring logic and cite them as the basis for your tropism dimension.

- Limitation in existing literature: No paper computationally scores AAV-disease fit across a landscape of diseases — that is your contribution.

2.3 Disease Similarity and Multi-Dimensional Scoring

These papers are from the gene prioritization and rare disease matching world, not gene therapy specifically — but they use scoring frameworks structurally identical to yours. The methodological sections are directly applicable.

2.3a DiSMVC — Multi-View Disease Similarity

- [DiSMVC \(BMC Bioinformatics, PMC11256965\)](#) — Graph collaborative learning framework measuring disease similarity across multiple biological feature views (genes, miRNAs). Read the Methods section — their feature weighting approach and how they validate similarity scores is directly applicable to validating your 6-dimension algorithm.
- Steal: Their validation approach — they test against known disease relationships as ground truth. You can do the same with known GT precedents (SMA → Zolgensma, HemB → Hemgenix) as your positive controls.

2.3b PatientMatcher — Python HPO Matching

- [PatientMatcher \(PMC9311682\)](#) — A Python-based open-source tool for matching rare disease patients via Orphanet, OMIM, DECIPHER. Uses HPO terms and phenotype scoring. Read this for how they structured their Python code, CLI, and scoring.
- Steal: Their code architecture — how they separated data fetching, matching, and output. Their scoring design choices (weighted vs unweighted, normalisation).

2.3c Phenotypic Similarity-Based Variant Prioritisation

- [Phenotypic Similarity for Rare Disease \(PMC10853199\)](#) — Uses HPO + ORDO + HOOM for three complementary similarity calculations. Excellent methods section on how they weight and combine different ontology-based dimensions.
- Steal: How they justify combining multiple similarity dimensions. Their Discussion section on limitations and what single-ontology approaches miss — frame your multi-dimension approach in contrast to this.

2.3d Large-Scale Computational Drug Repositioning

- [Large-scale computational drug repositioning \(PMC5847522\)](#) — Repositions drugs to rare diseases using multi-source scoring. The closest structural analogue to nanogt in the literature — they score drug-disease pairs across multiple dimensions and rank them.
- Steal: Their scoring normalisation approach, how they handle missing data across dimensions, how they present results as ranked lists with confidence, how they structure Discussion around limitations of static seed data.

2.4 Gene Therapy for Monogenic Rare Diseases — Reviews to Cite

These are the foundational biology papers your Introduction and Background should build from.

- [Gene Therapy for Monogenic Inherited Disorders \(PMC8045130\)](#) — Comprehensive review of challenges and opportunities. Cite for your motivation section.

- [Advances in Gene Therapy for Rare Diseases — MDPI 2026 \(PMC11765483\)](#) — 2026 review covering haploinsufficiency targeting, AAV, and mRNA. Very current. Useful for Introduction.
- [Accelerated Approval as the New Norm in GT for Rare Diseases \(2025\)](#) — Key regulatory context — the FDA accelerated approval pathway. Directly relevant to your regulatory precedent framing.
- [Microdystrophin as Surrogate Endpoint for DMD \(PMC10210223\)](#) — Perfect example of a surrogate endpoint paper — how one disease's molecular marker is validated as a regulatory surrogate. Exactly the intellectual move you are making computationally.

2.5 Orphanet and Rare Disease Data Infrastructure

- [Orphanet Representation in Health Information Systems](#) — The key Orphanet methodology paper. Cite this when describing how you use Orphanet data.
- [Orphanet Rare Disease Prevalence Analysis \(PMC7980535\)](#) — How to handle prevalence data from Orphanet — relevant if you include prevalence as a scoring dimension.
- [Aligning Orphanet Classification for Rare Disease Clusters \(PMC12422725\)](#) — Very recent (2026) analysis of Orphanet disease classification structure. Read to understand how your disease categories map onto Orphanet clusters.

2.6 Bioinformatics Tool Development — Code & Engineering Papers

- [Phen2Gene: Rapid Phenotype-Driven Gene Prioritization \(NAR Genomics, 2020\)](#) — Published Python/CLI rare disease tool. Median runtime 0.94 seconds, REST API, MIT license. Read this to understand what a published CLI bioinformatics tool looks like, how they document it, and how they benchmark performance.
- [Best Practices in Bioinformatics Software \(PMC4078907\)](#) — Foundational paper on what good bioinformatics software includes: tests, documentation, versioning, reproducibility. Use as a checklist for your own tool.
- [Bionitio: Demonstrating Best Practices for Bioinformatics CLI Software \(GigaScience 2019\)](#) — Practical guide to bioinformatics CLI tool design. Read for CLI structure, argument handling, logging, exit codes.

3. Scope — How Far Should You Go?

This is the most important strategic question. Here is the honest answer based on what published tools look like and what gets top marks in an MRes dissertation.

3.1 Number of Diseases

Recommendation: 15–25 diseases, not more

10 diseases is borderline — sufficient to demonstrate the tool works, but reviewers will ask 'why so few?' 25 is the sweet spot: enough to show patterns, cover different disease categories (metabolic, neuromuscular, haematological, lysosomal storage, etc.), and make statistically interesting comparisons — without becoming unmanageable. Avoid going beyond ~50 unless you automate the data collection fully, because the validity of each output still requires manual checking.

- **Must keep:** Your 5 current diseases (they include both positive controls and novel findings — critical)
- **Add ~10–15 more:** Aim for 3–4 diseases per major AAV-addressable category (liver, CNS, muscle, eye, haematopoietic). This lets you draw category-level conclusions.
- **Suggested additions:** MPS I (Hurler), MPS III type B, OTC deficiency, Pompe disease, Rett syndrome, Leber congenital amaurosis 2, spinal muscular atrophy type 3, PKU, Wilson disease, X-linked adrenoleukodystrophy
- **Justify your choices explicitly:** pick diseases where a GT program exists but may not be the obvious choice, and diseases with no current GT — that shows your tool's range

3.2 Scoring Dimensions — Should You Add More?

Your 6-dimension algorithm (max 10) is good. The question is whether the dimensions are the right ones and whether the weights are justified. This is where published papers can help.

Key gap to address

In your current algorithm, are weights arbitrary or literature-justified? PaVe-GT's Target Product Profiles and the surrogate endpoint literature give you explicit regulatory justification for which dimensions matter most (e.g., tissue tropism and gene size are hard constraints; immunogenicity is a major safety dimension). If you can cite papers for each dimension's weight, you go from 'I made a scoring system' to 'I implemented a literature-grounded scoring system.'

- **Dimensions almost certainly worth adding:** Patient population size (affects trial design), existing animal model availability (affects precedent strength), route of administration matching, COG status (for clinical priority)
- **Do not add dimensions just to add them:** Each must be justified. A 6-dimension well-justified score is stronger than a 10-dimension arbitrary one.
- **Consider sensitivity analysis:** Show what happens to rankings when you change weights by $\pm 20\%$. If your rankings are robust to weight changes, that is a strong validation. If they collapse, you need to rethink weights.

3.3 Should You Build a Website?

A website (e.g., a Flask/FastAPI web app or a Streamlit app) would significantly increase impact and demonstrably extend your tool beyond the CLI. It also maps onto what published bioinformatics tools look like — Phen2Gene has a web server, Exomiser has a web server, MotifPeeker has a documentation website.

Honest trade-off

A polished CLI + 20 diseases + robust validation + well-written dissertation will score better than a website + 10 diseases + thin validation. Only build a website if you can do it in under a week AND it does not compromise the write-up. A Streamlit app (Python, minimal extra code) that wraps your existing CLI is probably the right level of ambition if you do go this route.

- If you build a web interface: Streamlit is the fastest path — your existing Typer CLI functions can be called directly. Host on Streamlit Cloud (free).
- If you do not: Make sure your README and documentation are impeccable. A great README compensates for no website.

4. How to Write and Structure Your Project

4.1 Code Organisation — What Published Tools Do

Looking at MotifPeeker (R), Phen2Gene (Python/web), and PatientMatcher (Python) together, a clear pattern emerges for what published bioinformatics tools look like at the code level.

Element	What published tools do
Module separation	Separate files for: data fetching (disease.py, gene.py), scoring logic (scoring.py), output generation (report.py), CLI (cli.py). You already do this — good.
Config file	Scoring weights, API endpoints, DB paths in a config.py or config.yaml — not hardcoded. Makes parameter sensitivity analysis trivial.
Error handling	Explicit fallbacks for every API call (you have 12 gene fallbacks already). Log failures clearly. Never silently return None.
Unit tests	At minimum: test each scoring dimension with known inputs/outputs. Test your positive controls (SMA → Zolgensma score ~9.5). Published tools have test/ directories.
Logging	Use Python's logging module, not print(). INFO for normal flow, WARNING for fallbacks, ERROR for failures.
Requirements.txt / pyproject.toml	Pin exact dependency versions. Published tools are reproducible.
README	Install instructions, a worked example with real output, a figure showing the architecture, a citation.
Output formats	At minimum: Markdown report (you have this). Consider adding JSON output for programmatic use and a CSV summary for all diseases.

4.2 Dissertation Structure — How Published Papers Write This Up

Bioinformatics papers typically follow: Introduction → Methods → Results → Discussion → Conclusion. Your dissertation should follow this structure too, and here is what each section needs to contain at the level expected for an MRes.

Introduction (~1,500 words)

- The problem: >7,000 rare diseases, 95% without approved therapy, clinical development is slow and expensive
- The opportunity: Regulatory precedent from approved GTs can de-risk new programs — cite PaVe-GT
- The gap: No computational tool exists to systematically identify the best precedent for a given disease

- Your contribution: nanogt — what it does, how it works at a high level, your key result ('correctly identifies Zolgensma for SMA and Hemgenix for HemB as top-scoring precedents')
- End with a paragraph-level overview of the rest of the paper

Background (~2,000 words)

- Gene therapy biology: AAV vectors, serotypes, tropism — cite the Addgene guide + Nature STTT 2024 review
- Rare disease landscape: Orphanet, prevalence, monogenic diseases, the regulatory process — cite PaVe-GT + Brooks 2020
- Existing computational approaches: Disease similarity scoring, drug repurposing frameworks — cite DiSMVC, large-scale drug repositioning
- Why none of these solve your problem: They are not GT-specific, not regulatory-framed, not CLI tools — this is your niche

Methods (~3,000 words) — THE MOST IMPORTANT SECTION

This is where you will be marked hardest. The rule in bioinformatics methods is: someone else should be able to reproduce your results exactly. Every choice needs a citation or a justification.

- Data sources: Orphanet API (cite the Orphanet paper), UniProt (cite it), what fallbacks you use and why
- Database design: Your SQLite schema — why SQLite, what tables, what indexes
- Scoring algorithm: Each dimension defined precisely — formula, inputs, normalisation, score range, and why that weight. Reference papers for each dimension's scientific basis.
- Positive controls: Explicitly state you validated against SMA (Zolgensma) and HemB (Hemgenix) before running on novel diseases
- Novel outputs: Frame Fabry/MPS II finding as a hypothesis worth experimental validation, not a conclusion
- Software design: CLI interface, batch mode, report generation — cite Bionitio best practices paper

Results (~2,500 words)

- Present your disease-by-disease results with score breakdowns — use tables
- Group by disease category if you expand to 20+ diseases — show patterns
- Show sensitivity analysis on weights (even a simple $\pm 20\%$ run)
- Include one figure showing overall score distribution across all diseases and programs
- Compare your novel finding (Fabry/MPS II) to what a domain expert might have predicted

Discussion (~2,000 words)

- Interpret your results biologically, not just computationally

- Limitations: Static seed data (18 GT programs — small), no wet lab validation, scoring weights are literature-derived not optimised, no pharmacoeconomic dimension
- Future work: Expand catalog, add machine learning for weight optimisation, add web interface, validate novel predictions experimentally
- Comparison to existing tools: What you do that DiSMVC/drug-repurposing frameworks do not

5. What to Steal Right Now — Quick Wins

5.1 From MotifPeeker

- Their HTML report format: Rich, self-contained, shareable. Your Markdown reports are good but adding an HTML version (even via Jinja2 → browser) dramatically increases perceived impact.
- Their README structure: Badges, installation, usage, example outputs, citation. Copy this structure exactly.
- CI/CD with GitHub Actions: Even a basic workflow that runs your tests on push shows engineering maturity.

5.2 From the Drug Repurposing Papers

- Validation framework: Use your positive controls (SMA, HemB) as the benchmark set. Report precision at top-1 (does the tool return the correct approved therapy first?). This is a publishable validation metric.
- Sensitivity analysis: Change each dimension weight by $\pm 20\%$ and show ranking stability. Table format. This addresses the main reviewer question about your algorithm.

5.3 From PaVe-GT

- Target Product Profile concept: Map each of your 6 scoring dimensions to the components of a real TPP. This connects your algorithm to regulatory practice and is an incredibly strong framing device for your Discussion.
- Disease selection rationale: PaVe-GT chose diseases that share target organ, vector, and delivery route. Use this as your framework for justifying which diseases to expand to.

5.4 Limitations You Can Discuss That Others Have Too

Every good bioinformatics paper discusses its limitations honestly. Here are limitations that are (a) real, (b) acknowledged in the literature, and (c) that you can address or mitigate:

- Static database problem: Your 18 GT programs will be outdated. Mitigate by describing how the catalog can be updated.
- Weight optimisation: Your dimension weights are literature-derived, not learned from data. Frame as appropriate for a small dataset — note that machine learning would require a training set of hundreds of disease-program pairs, which does not yet exist.
- No wet-lab validation: Expected for a computational MRes. Frame novel predictions as testable hypotheses.
- Rare disease heterogeneity: Two diseases with the same Orphanet category can be biologically very different. Acknowledge this as a ceiling on scoring precision.

6. All Sources and Links

PaVe-GT / NIH

- [PaVe-GT Program Home](#) — NIH initiative for platform AAV gene therapy — your primary institutional precedent
- [Lomash et al. 2025 — AAV GT Early Planning](#) — Human Gene Therapy, PMC11971537
- [Brooks et al. 2020 — Platform Vector Gene Therapies Project](#) — Human Gene Therapy — the founding PaVe-GT concept paper
- [FDA Orphan Drug Navigation \(PaVe-GT, PMC10031144\)](#) — How they navigated ODD and RPDD designations

Gene Therapy Biology

- [AAV as delivery vector — Nature \(2024\)](#) — Signal Transduction & Targeted Therapy review
- [AAV Tropism Atlas 2025 — Molecular Therapy](#) — Comprehensive serotype-tissue mapping
- [Gene Therapy for Monogenic Inherited Disorders \(PMC8045130\)](#) — Challenges and opportunities review
- [Microdystrophin Surrogate Endpoint \(PMC10210223\)](#) — DMD surrogate endpoint precedent — regulatory framing
- [Accelerated Approval in GT for Rare Diseases 2025](#) — Regulatory landscape
- [Advances in GT for Rare Diseases \(PMC11765483\)](#) — 2026 review, MDPI

Disease Similarity and Scoring

- [DiSMVC Multi-View Disease Similarity \(PMC11256965\)](#) — Methods section for multi-dimensional disease scoring
- [Phenotypic Similarity Variant Prioritisation \(PMC10853199\)](#) — HPO + ORDO multi-dimensional approach
- [Large-Scale Computational Drug Repositioning \(PMC5847522\)](#) — Closest structural analogue to nanogt in literature
- [PatientMatcher Python CLI \(PMC9311682\)](#) — Python-based Orphanet/HPO matching tool — code architecture reference

Rare Disease Data and Orphanet

- [Orphanet Health Information Systems paper](#) — Primary citation for Orphanet data use
- [Orphanet API GitHub](#) — API documentation
- [Aligning Orphanet Classification \(PMC12422725\)](#) — 2026 analysis of Orphanet disease clusters

Bioinformatics Tool Engineering

- [Phen2Gene \(NAR Genomics 2020\)](#) — Published Python CLI rare disease tool — benchmark for what a published tool looks like
- [Best Practices in Bioinformatics Software \(PMC4078907\)](#) — Engineering checklist
- [Biontio CLI Best Practices \(GigaScience 2019\)](#) — CLI design guidance

- [MotifPeeker GitHub](#) — The other MRes project from your lab — published in Bioinformatics Advances

Prepared by Claude (Anthropic) for Lili Yassin | Imperial College London MRes Bioinformatics | June 2026